

Running an Uncensored Local LLM on an NVIDIA H200 — A Brief

Overview

This is a short, practical account of standing up large, self-hosted **open-weight LLMs** on a single **NVIDIA H200** GPU — models that, unlike filtered hosted assistants, will engage with the offensive-security

material a tester needs for **authorized** penetration testing and bug-bounty work. Everything runs locally:

no API keys, no per-token billing, and no scan data leaving infrastructure I control.

Why Local, and Why Uncensored

- **Control & privacy.** Target details, findings, and draft reports never leave the box.
- **No refusals on authorized work.** General-purpose hosted assistants routinely refuse legitimate exploitation questions even when you hold a signed scope document. Security-domain and refusal-removed

("ablated") open weights engage with the material instead.

- **Cost & latency.** A fixed GPU beats metered APIs for heavy, iterative use.
- **Key nuance:** capability is only half the story — it must be paired with authorization controls, or the

tool is a liability rather than an asset.

The Hardware

- **NVIDIA H200**, provisioned on NVIDIA Brev — **141 GB of VRAM**. That capacity is what makes running two

large models side by side possible.

The Serving Stack (and a Pivot)

- First attempt: **vLLM**. It crashed on the instance's bleeding-edge **CUDA 13.0 / Driver 580** stack — FlashAttention 3 was incompatible with the compiled kernels, and both the FlashInfer backend and enforce-eager mode died during CUDA graph capture.

- Pivot: **Ollama**, which handled the driver/runtime compatibility cleanly on the first try.
- **Takeaway:** on a day-zero CUDA stack, a packaged runtime (Ollama) beat a from-source server (vLLM).
- Added **Open WebUI** (a ChatGPT-style front end) on top of Ollama for interactive use.

The Models (Dual, Quantized)

- **WhiteRabbitNeo 33B** — a cybersecurity-specialized open-weight model for vulnerability analysis and

interactive security chat. GGUF `Q4_K_M`, ~19 GB.

- **Qwen2.5-72B-Instruct** (community "abliterated" build) — a strong generalist for clear, professional

report writing. GGUF `Q4_K_M`, ~47 GB.

- **Why it fits:** `Q4_K_M` 4-bit quantization shrinks ~100B parameters' worth of models to **~66 GB combined**, so a specialist and a generalist coexist on one GPU with headroom to spare.

- **A finding worth stating plainly:** a stock instruct model plus a "system-prompt override" still refuses

security questions — that behavior lives in the **weights**, not the prompt. The honest fix is choosing the

right weights, not fighting the wrong ones.

Quick Setup

```
# Install the runtime
curl -fsSL https://ollama.com/install.sh | sh

# Pull the models (GGUF Q4_K_M)
ollama pull <whiterabbitneo-33b-gguf>
ollama pull <qwen2.5-72b-abliterated-gguf>

# Optional: a ChatGPT-style UI on top
#   Open WebUI on port 8080, talking to Ollama on 11434
```

Key Numbers

Item	Value
GPU / VRAM	NVIDIA H200 · 141 GB
Driver / CUDA	580 / 13.0
Security model	WhiteRabbitNeo 33B · Q4_K_M · ~19 GB
Report model	Qwen2.5-72B abliterated · Q4_K_M · ~47 GB
Combined footprint	~66 GB (headroom on 141 GB)
Ports	Ollama 11434 · Open WebUI 8080

Lessons

1. On bleeding-edge CUDA, a packaged runtime can beat a from-source server.
2. `Q4_K_M` GGUF quantization is what lets ~100B params share one GPU.
3. Specialist + generalist beats one model doing everything.
4. Model refusal lives in the weights, not just the prompt.

Responsible Use

These models are self-hosted for **authorized security testing only** — systems you own or have explicit

written permission to assess. The capability exists to serve testers who already hold authorization; running

it against anything else is illegal and unethical.